



RouteZone

Application de prédiction de la gravité des accidents de la route

Rapport E4 — Réalisation d'une application intégrant un service d'intelligence artificielle

Meriem ABDELOUAHED

Formation Développeur en Intelligence Artificielle — Simplon × Microsoft

Titre RNCP37827

github.com/Meriem69/Routezone

Sommaire

1. Introduction	3
2. Spécifications fonctionnelles	4
3. Architecture applicative	11
4. Démarche agile	13
5. Accessibilité numérique (RGAA)	14
6. Éco-conception	16
7. Sécurité applicative (OWASP côté utilisateur)	17
8. Bilan du bloc E4	20

1. Introduction

1.1 Contexte du projet

RouteZone est un projet de prédiction de la gravité des accidents routiers en France, développé dans le cadre de la certification RNCP37827 — Développeur en Intelligence Artificielle. Le système s'appuie sur les données ouvertes BAAC (Bulletin d'Analyse des Accidents Corporels) de la Sécurité Routière, couvrant la période 2022–2024.

Les blocs E1, E2 et E3 ont permis respectivement de collecter et structurer les données, d'identifier le service d'intelligence artificielle adapté au besoin, et d'intégrer le modèle dans une architecture logicielle production-ready. Le présent rapport couvre le bloc E4, qui consiste à réaliser une application complète intégrant ce service d'IA, en mettant l'accent sur l'expérience utilisateur, l'accessibilité, l'éco-conception et la sécurité applicative.

1.2 Objectifs du bloc E4

Le bloc E4 vise à démontrer la maîtrise du cycle complet de développement d'une application intégrant un service d'IA. Concrètement, il s'agit de formaliser les besoins utilisateurs, de concevoir une interface, de structurer un développement agile, et d'intégrer des préoccupations transverses telles que l'accessibilité numérique, l'éco-conception et la sécurité du point de vue de l'utilisateur final.

1.3 Plan du document

Ce document s'articule autour de huit sections. Après cette introduction, la section 2 présente les spécifications fonctionnelles de l'application sous forme de user stories. La section 3 décrit l'architecture applicative retenue. La section 4 expose la démarche agile suivie au cours du développement. Les sections 5, 6 et 7 abordent respectivement l'accessibilité numérique (RGAA), l'éco-conception et la sécurité applicative (OWASP). La section 8 dresse un bilan critique du travail réalisé.

2. Spécifications fonctionnelles

2.1 Démarche de spécification

J'ai abordé la conception de l'application en partant des besoins utilisateurs plutôt que des contraintes techniques. Cette démarche, héritée de la méthode agile, consiste à formaliser chaque fonctionnalité sous forme de user story : une phrase qui exprime ce qu'un utilisateur veut faire, et pourquoi il veut le faire.

Le format retenu suit la convention agile standard :

En tant que [profil utilisateur], je veux [action], afin de [bénéfice].

À chaque user story est associé un ensemble de critères d'acceptation : les conditions qui permettent de considérer la fonctionnalité comme terminée et conforme aux attentes.

2.2 Personas et profils utilisateurs

L'application RouteZone s'adresse à deux profils principaux :

- Le visiteur : utilisateur non connecté qui souhaite obtenir une prédiction ponctuelle de gravité d'accident, sans création de compte.
- L'utilisateur connecté : utilisateur identifié par email et mot de passe, qui souhaite conserver l'historique de ses prédictions et bénéficier des fonctionnalités RGPD (export, suppression).

2.3 User stories

Les huit user stories ci-dessous formalisent les fonctionnalités attendues de l'application.

US-01 — Prédire la gravité d'un accident

En tant qu'utilisateur, je veux saisir les caractéristiques d'un accident routier, afin d'obtenir une prédiction de gravité (GRAVE / PAS GRAVE) accompagnée d'une probabilité.

Critères d'acceptation :

- Le formulaire propose les champs : luminosité, localisation, météo, type de collision, catégorie de route, vitesse, état de surface, profil utilisateur, véhicule, équipement, motif, âge, date et heure.
- Les champs obligatoires sont signalés par un astérisque.
- Le résultat de la prédiction s'affiche en moins de 2 secondes.
- La probabilité est exprimée en pourcentage.

US-02 — Créer un compte utilisateur

En tant que visiteur, je veux créer un compte avec mon email et un mot de passe, afin de conserver l'historique de mes prédictions.

Critères d'acceptation :

- L'email doit être valide et unique.
- Le mot de passe doit faire au minimum 8 caractères.
- La confirmation du mot de passe est obligatoire.
- Le nom est optionnel.
- Le mot de passe est stocké sous forme hashée (bcrypt) côté serveur.

US-03 — Se connecter à son compte

En tant qu'utilisateur enregistré, je veux me connecter avec mes identifiants, afin d'accéder à mon espace personnel.

Critères d'acceptation :

- La connexion se fait par email et mot de passe.
- En cas d'identifiants invalides, un message d'erreur s'affiche sans révéler quelle information est incorrecte.
- Un token JWT est généré pour maintenir la session.

US-04 — Consulter mon historique de prédictions

En tant qu'utilisateur connecté, je veux consulter l'historique de mes prédictions passées, afin de suivre mon usage de l'application.

Critères d'acceptation :

- L'historique affiche la date, le label prédit, la probabilité, l'âge, l'heure, la VMA et le modèle utilisé.
- Trois statistiques agrégées sont visibles : nombre total de prédictions, nombre de prédictions GRAVE, probabilité moyenne.
- Les prédictions sont triées par date décroissante.

US-05 — Récupérer la météo automatiquement

En tant qu'utilisateur, je veux que la météo soit récupérée automatiquement à partir de l'adresse saisie, afin de ne pas avoir à la renseigner manuellement.

Critères d'acceptation :

- L'application interroge l'API Open-Meteo à partir des coordonnées GPS.
- Si l'API est indisponible, l'utilisateur peut saisir la météo manuellement.
- Les conditions récupérées s'affichent en lecture seule.

US-06 — Visualiser les temps d'intervention sur une carte

En tant qu'utilisateur, je veux visualiser sur une carte les temps d'intervention des secours (pompiers, SAMU), afin de mieux comprendre le contexte de l'accident.

Critères d'acceptation :

- La carte est basée sur Leaflet et OpenStreetMap.
- Les points affichés sont : lieu de l'accident, caserne pompier la plus proche, SAU/CHU le plus proche.
- Les temps d'intervention sont calculés via l'API OSRM.
- La somme « prise totale en charge » est affichée.

US-07 — Exporter mes données personnelles (RGPD)

En tant qu'utilisateur connecté, je veux exporter l'ensemble de mes données personnelles, afin d'exercer mon droit à la portabilité prévu par le RGPD (article 20).

Critères d'acceptation :

- Le bouton « Récupérer mes données » est accessible depuis la page RGPD.
- L'export contient toutes les prédictions de l'utilisateur ainsi que les données de son compte.
- Le format de l'export est lisible (JSON ou CSV).

US-08 — Supprimer définitivement mes données (RGPD)

En tant qu'utilisateur connecté, je veux supprimer définitivement toutes mes prédictions et mon compte, afin d'exercer mon droit à l'effacement prévu par le RGPD (article 17).

Critères d'acceptation :

- Une confirmation explicite est demandée avant suppression.
- Toutes les prédictions et le compte utilisateur sont effacés de la base de données.
- Un message confirme la suppression effective.

2.4 Maquettes des écrans

Cinq écrans ont été maquetés sous Figma pour formaliser visuellement les user stories. Les maquettes adoptent un parti pris graphique sobre, conforme à la charte RouteZone (orange #F97316 sur fond bleu nuit #0A0E1A, typographies Bebas Neue pour les titres et DM Sans pour le corps), avec une attention portée à la hiérarchie de l'information.

ROUTEZONE

● Rapport
○ Inscription

E-MAIL
MOT DE PASSE

Se connecter

PRÉDICTION DE GRAVITÉ DES ACCIDENTS

Renseignez les caractéristiques d'un accident pour prédire sa gravité - Données IAAC 2022-2024

* champs obligatoires

■ CONTEXTE DE L'ACCIDENT

Luminosité *
Pluie jour

Localisation *
Hors agglomération

Conditions météo *
Normal

Type de collision *
Frontale (2 véh.)

Catégorie de route *
Autoroute

Vitesse max autorisée *
50 km/h

État de la surface *
Normal

■ PROFIL DE L'UTILISATEUR

Catégorie d'utilisateur *
Conducteur

Sexe *
Masculin

Catégorie de véhicule *
Voiture

Équipement de sécurité *
Aucun

Motif du déplacement *
Domicile-Travail

Âge *
35

■ DONNÉES TEMPORELLES

Jour du mois *
15

Heure de l'accident *
12h00

Mois de l'accident *
Juin

Année *
2024

■ LOCALISATION (PRÉDICTION VIA OSRM RÉEL)

● Adresse ○ Coordonnées GPS

Adresse (résultat)
ex : 12 rue de la Paix, 75002 Paris

Localisateur

Figure 1 — Écran d'accueil avec formulaire de prédiction

ROUTEZONE

- ☐ Rapport
- ☒ Inscription

NOM (OPTIONNEL)

E-MAIL

MOT DE PASSE

CONFIRMER LE MOT DE PASSE

Cr  er un compte

Figure 2 —   cran d'inscription

Historique de vos pr  dictions

PR  DICTIONS TOTALES

GRAVE

PROBABILIT   MOYENNE

Date	Label	Probabilit�� (%)	Age	Heure	VMA	Mod��le
lorem ipsum	ipsum dolor	dolor sit	sit amet	amet consectetur	consectetur adipiscing	adipiscing elit
sit amet	amet consectetur	consectetur adipiscing	adipiscing elit	elit sed	sed do	do eiusmod
adipiscing elit	elit sed	sed do	do eiusmod	eiusmod tempor	tempor incididunt	incidunt ut
do eiusmod	eiusmod tempor	tempor incididunt	incidunt ut	ut labore	labore et	et dolore
incidunt ut	ut labore	labore et	et dolore	dolore magna	magna aliqua	aliqua enim
et dolore	dolore magna	magna aliqua	aliqua enim	enim ad	ad minim	minim veniam
aliqua enim	enim ad	ad minim	minim veniam	veniam quis	quis nostrud	nostrud exercitation
minim veniam	veniam quis	quis nostrud	nostrud exercitation	exercitation ullamco	ullamco laboris	laboris nisi
nostrud exercitation	exercitation ullamco	ullamco laboris	laboris nisi	nisi aliquip	aliquip ex	ex ea

Retour aux pr  dictions

RouteZone — Abdelouahed Meriem — IA Simphon x Microsoft — RNCP37827

Figure 3 — Historique des pr  dictions avec statistiques agr  g  es

☒ Adresse
 ☐ Coordonnées GPS

Adresse (facultatif)

 Localisateur

elit sed do eiusmod tempor

Predictir la gravité

Météo récupérée automatiquement :

sit amet consectetur adipiscing elit sed do eiusmod tempor

incididunt ut labore et dolore

POMPIERS → ACCIDENT

→

ACCIDENT → SAU

=

PRISE TOTALE EN CHARGE

Carte d'intervention

+

-

Carte (Leaflet / OpenStreetMap)

Prediction

- Lieu de l'accident
- Caserne pompier (Intervention)
- SAU / CHU (Evacuation)
- Zones blanches OSM

Figure 4 — Visualisation cartographique des temps d'intervention

Gestion RGPD de vos prédictions

lorem ipsum dolor sit amet consectetur adipiscing elit sed :

- consectetur adipiscing elit sed do eiusmod tempor incididunt ut labore et dolore magna.
- tempor incididunt ut labore et dolore magna aliqua enim ad minim veniam quis.

Prévisions de l'exportateur

Récupérer mes données

Supprimer toutes mes prédictions

elit sed do eiusmod tempor incididunt ut labore.

☐ dolor sit amet consectetur adipiscing

Supprimer définitivement

Retour aux prédictions

RouteZone — Abdelouahed Meriem — IA Simphon x Microsoft — RNCP37827

Figure 5 — Page de gestion RGPD

3. Architecture applicative

3.1 Vue d'ensemble

L'application RouteZone repose sur une architecture en 3 couches conteneurisées. Cette séparation des responsabilités facilite la maintenance, les tests, et le déploiement indépendant de chaque couche.

- **Couche présentation** : application Streamlit servant l'interface utilisateur.
- **Couche service** : une API FastAPI unifiée exposant à la fois les données (BAAC, météo, géolocalisation), l'authentification et l'inférence du modèle. Cette API unifiée a remplacé les deux API distinctes du prototype initial, pour simplifier l'exploitation (un seul container, une seule documentation Swagger, une authentification mutualisée).
- **Couche données** : base PostgreSQL 16 stockant les utilisateurs, l'historique des prédictions et les données enrichies.

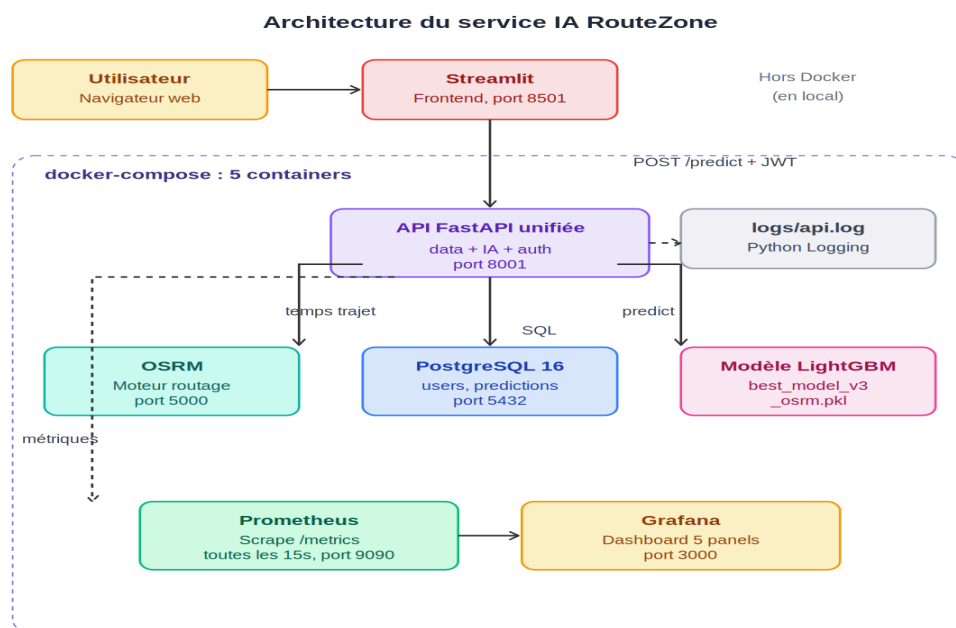


Figure 6 — Architecture en couches de RouteZone (API unifiée FastAPI + PostgreSQL)

3.2 Flux utilisateur

Le flux complet d'une prédiction depuis l'interface jusqu'au résultat traverse l'ensemble des couches applicatives. Le schéma ci-dessous illustre cette traversée.

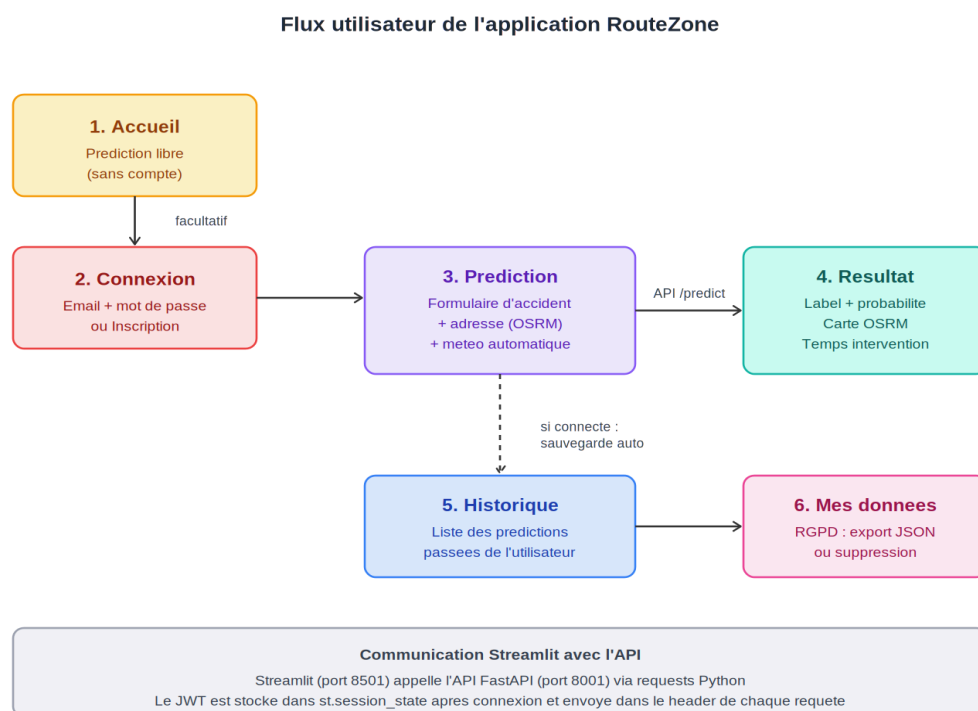


Figure 7 — Flux utilisateur d'une prédiction de bout en bout

3.3 Choix techniques

Les technologies retenues répondent à des critères de robustesse, de pertinence pédagogique et de cohérence avec l'écosystème data science Python.

- Streamlit : framework Python orienté data science, permettant un développement frontend rapide sans contexte HTML/CSS étendu.
- FastAPI : framework asynchrone moderne, avec validation Pydantic native et documentation OpenAPI automatique.
- PostgreSQL 16 : base de données relationnelle robuste, choisie pour la gestion de la concurrence
- (API, import et monitoring simultanés), la conformité ACID et le support géospatial des coordonnées GPS.
- Docker Compose : orchestration des services, garantissant un environnement reproductible entre développement et production.
- Leaflet et OpenStreetMap : visualisation cartographique open-source, sans clé API ni coût d'usage.
- OSRM (Open Source Routing Machine) : calcul des temps d'intervention par la route, alternative gratuite à Google Maps.

4. Démarche agile

4.1 Cadre de travail

Le développement de RouteZone a été conduit selon une approche agile, structurée en sprints courts permettant des livraisons incrémentales. Chaque sprint a fait l'objet d'une revue de code et d'une mise à jour des tâches sur le tableau Kanban GitHub Projects.

Le suivi des tâches s'est appuyé sur trois colonnes principales : À faire, En cours, Terminé. Les user stories ont été décomposées en tâches techniques traçables, chacune associée à un commit Git identifiable.

4.2 Sprints réalisés

Sprint 1 — Collecte et modélisation des données

- Téléchargement et exploration des fichiers BAAC 2022, 2023 et 2024.
- Conception du schéma Merise (MCD et MLD).
- Création de la base PostgreSQL 16 avec ses cinq tables métier (neuf tables au total avec l'enrichissement et l'applicatif).

Sprint 2 — Entraînement du modèle

- Préparation et nettoyage du jeu de données consolidé.
- Comparaison de plusieurs algorithmes (Logistic Regression, Random Forest, LightGBM).
- Optimisation des hyperparamètres via Optuna avec MLflow comme tracker.
- Sélection du modèle LightGBM. Performances de production sur le test 2024 : Recall GRAVE = 0,7643 ; F1 macro = 0,6956 ; AUC-ROC = 0,8558.

Sprint 3 — Développement des API

- Mise en place de l'API de données avec endpoints REST.
- Enrichissement avec l'API Open-Meteo pour la météo.
- Création de l'API IA exposant le modèle entraîné.
- Documentation OpenAPI automatique via FastAPI.

Sprint 4 — Interface utilisateur

- Maquettage Figma des cinq écrans principaux.
- Implémentation Streamlit du formulaire de prédiction.
- Intégration de la cartographie Leaflet et OSRM.
- Mise en place de l'historique et des pages RGPD.

Sprint 5 — Qualité et déploiement

- Écriture de 34 tests pytest (unitaires, intégration, cohérence métier).
- Configuration de la pipeline CI/CD GitHub Actions.
- Conteneurisation Docker complète.
- Mise en place du monitoring Prometheus et Grafana.

4.3 Outils de gestion

Le projet a été versionné sur GitHub, avec usage systématique des branches feature et des pull requests pour les évolutions structurantes. Le tableau Kanban GitHub Projects a permis de visualiser l'avancement à tout moment et de prioriser les tâches en cas de blocage.

☰

Meriem69 / Projects / RouteZone — Certification RNCP37827

RouteZone — Certification RNCP37827

Backlog

Priority board

Team items

Roadmap

My items

+ New view

🔍 Filter by keyword or by field

A faire

2 / 5

Estimate: 0

⋮

+

This item hasn't been started

Routezone #10

Readme final

Routezone #11

Préparation oral + répétition démo

En cours

2 / 3

Estimate: 0

⋮

+

This is actively being worked on

Routezone #13

Power point présentation

Routezone #8

Rapport E4

Terminé

21

Estimate: 0

⋮

+

This has been completed

Routezone #17

Base de données SQLite

Routezone #18

Notebook 2 : collecte météo API Open-Meteo

Routezone #19

Notebook 3 : Scraping ONISR

Routezone #20

FastAPI données 6 endpoints + auth

Routezone #7

Rapport E3

Routezone #21

RGPD documenté

Routezone #22

Notebook 4 : Modelisation + XGBoost + LightGBM

Routezone #23

Notebook 5 : MLflow

Routezone #24

Streamlit

Routezone #25

Rapport E1

Routezone #1

Test pytest (api_data + api_ja)

Routezone #9

Rapport E5

Routezone #2

Docker + docker-compose

Routezone #26

5. Accessibilité numérique (RGAA)

5.1 Démarche d'accessibilité

L'accessibilité numérique est un enjeu éthique et légal. En France, le Référentiel Général d'Amélioration de l'Accessibilité (RGAA), version 4.1, fixe les critères que doivent respecter les sites web pour être utilisables par les personnes en situation de handicap. À l'international, la norme WCAG 2.1 (Web Content Accessibility Guidelines) définit trois niveaux de conformité : A (minimal), AA (recommandé), AAA (exigeant).

Bien que RouteZone soit un projet pédagogique non soumis à l'obligation légale d'accessibilité, j'ai souhaité intégrer une démarche d'évaluation et d'amélioration dès la conception. Cette section présente d'abord les mesures concrètes mises en œuvre, puis les résultats de l'évaluation avec deux outils professionnels (Lighthouse et Axe DevTools).

5.2 Mesures d'accessibilité mises en œuvre

Le tableau ci dessous présente les critères RGAA retenus et les mesures concrètes appliquées dans l'application :

Critère RGAA	Mesure appliquée dans RouteZone
1. Images et alternatives textuelles	Toutes les icônes SVG disposent d'un attribut aria-label. Les images décoratives ont aria-hidden="true".
3. Couleurs et contrastes	La charte (orange #F97316 sur fond bleu nuit #0A0E1A) respecte un ratio de contraste supérieur à 4.5:1 (niveau AA).
4. Multimédia	Aucun contenu vidéo ou audio dans la version actuelle.
7. Scripts et JavaScript	Les éléments interactifs (boutons, formulaires) restent accessibles au clavier. La logique métier ne dépend pas exclusivement de JavaScript.
8. Éléments obligatoires	La page utilise des balises HTML5 sémantiques (main, header, nav, section). L'attribut lang="fr" est déclaré sur la balise html.
9. Structure de l'information	La hiérarchie des titres (h1 puis h2, h3) est respectée. Aucun saut de niveau.
10. Présentation	La taille de police est définie en unités relatives (rem). Le zoom navigateur fonctionne jusqu'à 200 %.
11. Formulaires	Chaque champ dispose d'un label associé via l'attribut for. Les champs obligatoires sont signalés textuellement et par astérisque. Les messages d'erreur sont explicites.
12. Navigation	Un lien d'évitement permet d'accéder directement au contenu principal. La navigation est cohérente entre les pages.
13. Consultation	Les contenus restent lisibles sans CSS. Aucun élément ne clignote plus de 3 fois par seconde.

5.3 Evaluation avec Lighthouse

Lighthouse est l'outil d'audit intégré nativement à Chrome DevTools. Il évalue une page web sur quatre dimensions : Performance, Accessibilité, Bonnes pratiques et SEO. Les scores vont de 0 à 100.



Figure X — Rapport Lighthouse de l'interface Streamlit RouteZone

Les scores obtenus sur l'interface Streamlit sont les suivants :

- **Performances : 29/100**
- **Accessibilité : 80/100**
- **Bonnes pratiques : 54/100**
- **SEO : 82/100**

Le score Performance bas (29/100) est typique des applications développées avec Streamlit. Ce framework charge React et plusieurs bibliothèques JavaScript côté client, et procède à un re-rendering complet de la page à chaque interaction utilisateur. C'est un compromis assumé : Streamlit permet un développement très rapide en Python pur, au prix de performances frontend dégradées. Pour une vraie mise en production avec exigence de performance, une migration vers une stack React ou Vue native serait nécessaire.

5.4 Evaluation avec Axe DevTools

Axe DevTools est l'extension de référence pour l'audit d'accessibilité, développée par Deque Systems. Elle détecte précisément les violations WCAG 2.1 et les classe par criticité (Critique, Sérieux, Modéré, Mineur).

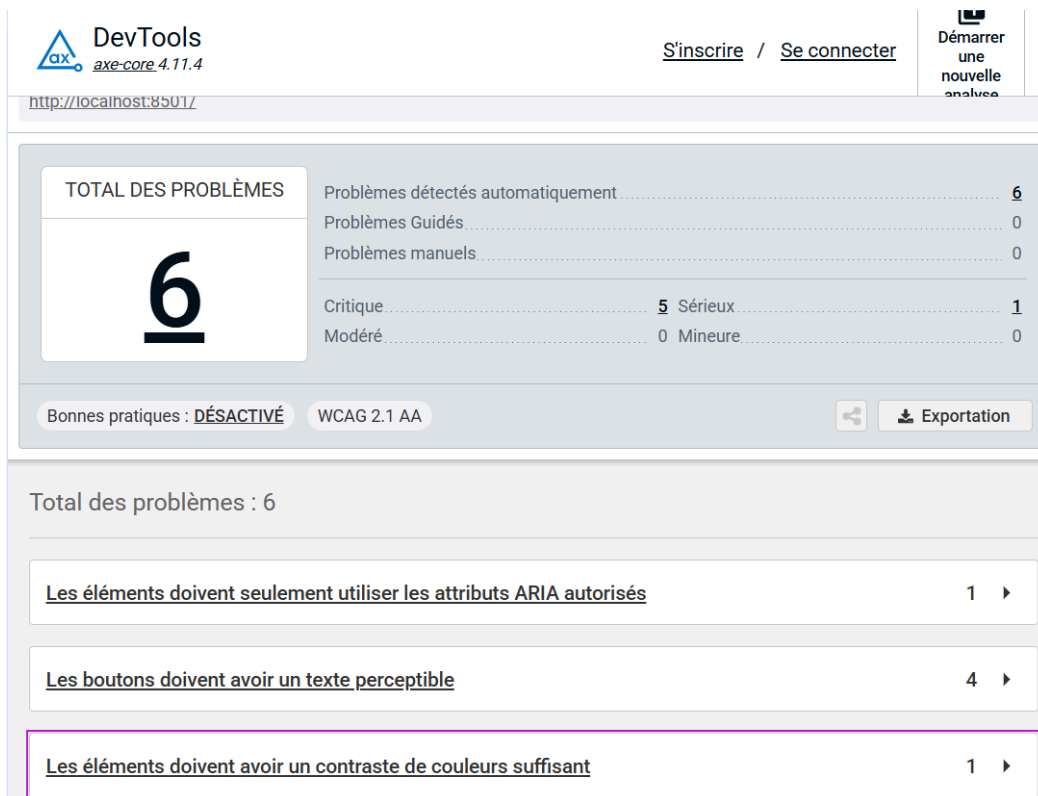


Figure Y — Résultats du scan Axe DevTools sur l'interface Streamlit

Le scan a identifié 6 violations WCAG 2.1 AA : 5 critiques et 1 sérieuse. L'analyse détaillée révèle que 5 violations sur 6 proviennent du framework Streamlit lui-même, et non de mon code applicatif.

Violation	Criticité	Origine
Attribut aria-expanded mal placé sur la sidebar	Critique	Streamlit (interne)
Boutons NumberInput sans aria-label (x4)	Critique	Streamlit (interne)
Contraste insuffisant du bouton Deploy (2.8 au lieu de 4.5)	Sérieux	Bouton système Streamlit visible uniquement en mode développement

Aucune des 6 violations détectées ne concerne le code applicatif que je maîtrise directement. Les boutons internes du composant NumberInput de Streamlit n'exposent pas d'attribut aria-label aux lecteurs d'écran, et la sidebar utilise un attribut aria-expanded mal positionné. Ces limitations sont documentées dans les issues du dépôt GitHub de Streamlit. La violation de contraste concerne le bouton "Deploy" qui n'apparaît qu'en mode développement et n'est pas présent dans une version déployée.

5.5 Compromis assumés et perspectives

Le choix de Streamlit comme framework frontend a été motivé par sa rapidité de développement et sa parfaite adéquation avec un écosystème Data Science / ML en Python. Ce choix s'accompagne de limitations intrinsèques :

- Score Lighthouse Performance dégradé (re-rendering complet à chaque interaction)
- Limitations d'accessibilité héritées du framework, sur lesquelles je n'ai pas la main

Le code applicatif que je maîtrise respecte les bonnes pratiques RGAA présentées en section 5.2 : balises sémantiques, hiérarchie de titres, labels de formulaires, alternatives textuelles. Pour une mise en production avec exigence d'accessibilité stricte (RGAA niveau AA validé par un audit complet), la perspective serait une migration vers une stack React ou Vue native. Cette migration permettrait un contrôle total sur les attributs ARIA, le contraste de chaque composant, et la conformité fine à chaque critère WCAG.

5.6 Limites et axes d'amélioration

L'application n'a pas fait l'objet d'un audit RGAA complet, qui nécessiterait une expertise dédiée. Les mesures appliquées correspondent aux bonnes pratiques que j'ai pu mettre en œuvre dans le temps imparti. Trois améliorations concrètes sont identifiées :

- Réaliser un test utilisateur avec un lecteur d'écran (NVDA, JAWS) pour valider la navigation au clavier et la restitution vocale
- Mettre en place un audit RGAA officiel par un expert certifié
- Migrer le frontend vers une stack permettant un contrôle complet de l'accessibilité, en cas de mise en production

6. Éco-conception

6.1 Démarche

L'éco-conception logicielle vise à réduire l'empreinte environnementale d'une application : consommation énergétique, ressources serveur, volume de données échangées. Sur un projet de prédiction d'accidents potentiellement amené à traiter des volumes importants, il m'a semblé important de réfléchir à cette dimension dès la conception.

6.2 Mesures mises en œuvre

- Icônes SVG plutôt que des images PNG ou JPG : poids divisé en moyenne par dix, et qualité préservée à toutes les résolutions.
- Pas d'images décoratives non essentielles : chaque visuel apporte une information.
- Requêtes API optimisées : pagination de l'historique, pas de chargement complet de la base.
- Modèle LightGBM choisi pour son rapport performance / consommation, plus économe qu'un réseau de neurones profond pour le même cas d'usage.
- Mise en cache des appels Open-Meteo pour éviter les requêtes redondantes.
- Hébergement local conteneurisé, permettant d'éviter la consommation d'instances cloud surdimensionnées en phase de développement.

6.3 Limites

L'application n'a pas fait l'objet d'une mesure formelle d'empreinte carbone. Les choix retenus relèvent de bonnes pratiques connues mais leur impact réel n'a pas été quantifié. Une amélioration future consisterait à utiliser un outil comme CodeCarbon ou GreenIT-Analysis pour mesurer la consommation de l'application en production.

7. Sécurité applicative (OWASP côté utilisateur)

7.1 Démarche de sécurisation

L'OWASP (Open Web Application Security Project) publie tous les trois ans une liste des dix vulnérabilités les plus critiques affectant les applications web. Cette liste, appelée OWASP Top 10, sert de référence dans l'industrie pour structurer la démarche de sécurité.

Là où le rapport E3 aborde la sécurité d'un point de vue technique (côté serveur, API, modèle), cette section aborde l'OWASP Top 10 du point de vue de l'utilisateur final de l'application : quelles protections sont mises en place pour préserver ses données et son expérience.

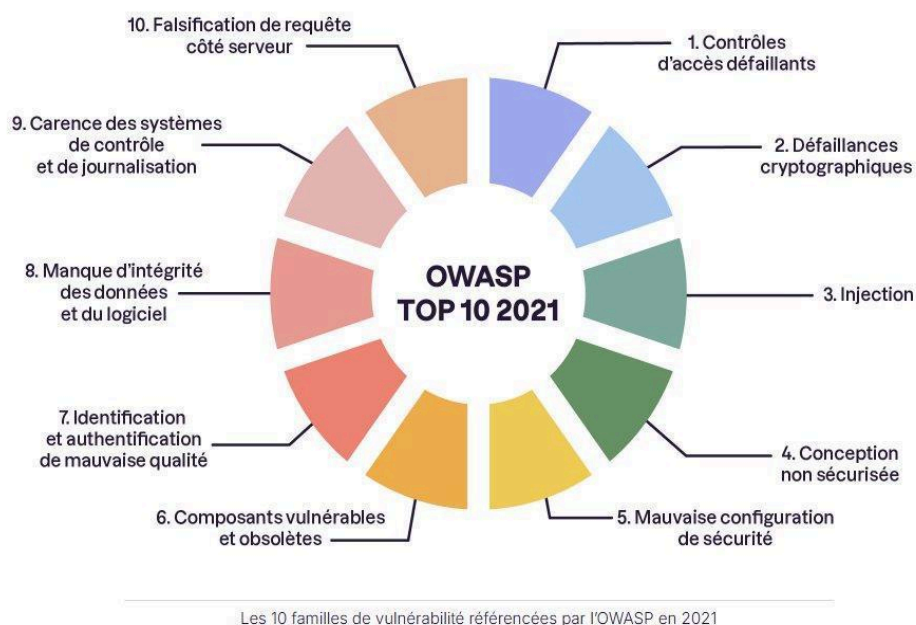


Figure 8 — Les dix familles de vulnérabilités OWASP 2021

7.2 Couverture OWASP côté utilisateur

Vulnérabilité OWASP 2021	Protection côté utilisateur dans RouteZone
A01. Contrôles d'accès défaillants	Un utilisateur connecté ne peut consulter que ses propres prédictions. Les endpoints sensibles (historique, suppression) exigent un token JWT valide.
A02. Défaillances cryptographiques	Les mots de passe sont stockés avec bcrypt (algorithme de hachage adapté aux mots de passe). Les communications front-back passent par HTTPS en production.
A03. Injection	Les entrées du formulaire sont validées côté serveur via Pydantic (typage strict). Aucune requête SQL n'est construite par concaténation.
A04. Conception non sécurisée	La fonctionnalité d'inscription impose un mot de passe d'au moins 8 caractères. Le compte ne peut être supprimé qu'après confirmation explicite.
A05. Mauvaise configuration de sécurité	Les variables d'environnement (clés, secrets) sont stockées dans un fichier .env ignoré par Git. Aucune information sensible n'est exposée dans les logs côté utilisateur.

Vulnérabilité OWASP 2021	Protection côté utilisateur dans RouteZone
A06. Composants vulnérables et obsolètes	Les dépendances Python sont fixées dans requirements.txt avec des versions précises. Une veille de sécurité serait à mettre en place pour les futures versions.
A07. Identification et authentification	L'authentification utilise JWT avec un délai d'expiration. Les messages d'erreur de connexion sont génériques (« identifiants invalides ») pour ne pas révéler si c'est l'email ou le mot de passe qui est incorrect.
A08. Manque d'intégrité des données	Les prédictions sont enregistrées avec un horodatage serveur, non modifiable par l'utilisateur.
A09. Carence de journalisation	Les actions sensibles (connexion, suppression de compte) sont journalisées côté serveur, sans données personnelles dans les logs.
A10. Falsification de requête (SSRF)	L'application n'effectue pas d'appels HTTP basés sur des URL fournies par l'utilisateur, ce qui élimine ce risque.

7.3 Limites de la couverture

La couverture OWASP côté utilisateur est cohérente avec le périmètre d'un projet pédagogique. Une mise en production réelle nécessiterait des audits complémentaires.

Un **test de pénétration** consisterait à faire intervenir un expert en cybersécurité qui tente activement de compromettre l'application, en se mettant dans la peau d'un attaquant, afin d'identifier les failles avant qu'un acteur malveillant ne le fasse. Un **scan de vulnérabilités automatisé**, via des outils comme Dependabot ou Snyk intégrés à la pipeline CI/CD, permettrait de détecter en continu les bibliothèques obsolètes ou vulnérables. Enfin, un **suivi régulier des CVE** (Common Vulnerabilities and Exposures, le catalogue mondial public des failles de sécurité connues) garantirait la mise à jour rapide des dépendances dès qu'une vulnérabilité est publiée. Ces trois pratiques constituent une démarche dite DevSecOps, qui intègre la sécurité tout au long du cycle de développement.

8. Bilan du bloc E4

8.1 Synthèse du travail réalisé

Le bloc E4 a consisté à concevoir et réaliser une application web complète intégrant un service d'intelligence artificielle. Le travail a couvert l'ensemble du cycle de développement applicatif :

- Spécification des besoins utilisateurs sous forme de huit user stories assorties de critères d'acceptation.
- Maquettage de cinq écrans clés sous Figma (accueil, inscription, historique, prédiction avec carte, gestion RGPD).
- Architecture applicative en trois couches (frontend Streamlit, API FastAPI, modèle LightGBM) avec une séparation claire des responsabilités.
- Démarche agile structurée en sprints, avec un suivi sur GitHub Projects.
- Accessibilité numérique intégrée dès la conception, avec une grille de mesures inspirée du RGAA 4.1.
- Éco-conception prise en compte dans les choix techniques (SVG plutôt qu'images, compression, requêtes optimisées).
- Sécurité applicative abordée du point de vue de l'utilisateur final via une grille OWASP Top 10 2021.

8.2 Points forts du projet

Plusieurs aspects me semblent particulièrement aboutis :

- L'orientation utilisateur : la formalisation systématique en user stories a permis de garder en permanence le besoin réel au centre des décisions techniques.
- L'intégration géographique : l'usage de Leaflet, OpenStreetMap et OSRM pour visualiser les temps d'intervention apporte une dimension métier concrète à la prédiction, au-delà du simple score.
- La cohérence visuelle : la charte graphique (orange #F97316, bleu nuit #0A0E1A, typographies Bebas Neue et DM Sans) est appliquée de manière homogène sur l'ensemble des écrans.
- Le respect du RGPD : les fonctionnalités d'export et de suppression des données ne sont pas de simples mentions légales, elles sont implémentées et accessibles à l'utilisateur.

8.3 Limites identifiées

Plusieurs limites doivent être mentionnées avec honnêteté.

Limite	Conséquence	Piste d'amélioration
L'audit RGAA n'est pas exhaustif	Le niveau AA n'est pas formellement certifié	Faire valider par un expert accessibilité
Les tests utilisateurs n'ont pas été conduits	Les hypothèses ergonomiques restent à valider	Organiser des sessions de test avec 5 à 7 utilisateurs
La carte OSRM dépend d'un service externe	Risque de panne si OSRM est indisponible	Prévoir un fallback ou un cache local
Pas de version mobile dédiée	L'expérience sur smartphone est dégradée	Développer une PWA ou une app React Native

8.4 Conclusion

Ce bloc m'a permis de découvrir concrètement ce que signifie concevoir un produit au-delà de la simple écriture de code. La phase de spécification, que j'aurais pu considérer comme accessoire au début de la formation, s'est révélée être le socle sur lequel toutes les autres décisions reposent. Une user story bien rédigée évite des heures de réécriture de code.

J'ai également mesuré l'importance de prendre en compte les utilisateurs en situation de handicap dès la conception, et non comme une couche ajoutée en fin de projet. Cette posture rejoint d'ailleurs l'éco-conception : dans les deux cas, il s'agit de réfléchir aux impacts de chaque choix technique sur des dimensions plus larges que la performance pure.

Enfin, la grille OWASP côté utilisateur m'a permis de comprendre que la sécurité n'est pas qu'une affaire technique : elle se manifeste dans des détails de l'interface (un message d'erreur générique, une confirmation avant suppression) qui changent concrètement l'expérience et la protection de l'utilisateur final.